Job Market Analytics Dashboard

Step 5 — Dashboard Route & Data Rendering

Step 5 Overview

Status: ✓ Complete

Updated app.py to query Supabase and compute analytics (total jobs, average salary, top skills, top locations), then pass all data to dashboard.html. Updated the template to render real data from the database. Confirmed full data pipeline working end to end in the browser.

5.1 What Changed in app.py

Added Supabase client and updated the dashboard() route:

```
from flask import Flask, render_template
from supabase import create_client
from dotenv import load_dotenv
import os
load_dotenv()
app = Flask(__name__)
supabase = create_client(
os.getenv("SUPABASE_URL"),
os.getenv("SUPABASE_KEY")
)
@app.route("/")
def dashboard():
# fetch all jobs from Supabase
result = supabase.table("jobs").select("*").execute()
jobs = result.data
# total count
total_jobs = len(jobs)
# top skills using Counter
all_skills = []
for job in jobs:
all_skills.extend(job.get("skills") or [])
from collections import Counter
skill_counts = Counter(all_skills).most_common(10)
# average salary
salaries = [j["salary_min"] for j in jobs if j.get("salary_min")]
avg_salary = round(sum(salaries) / len(salaries)) if salaries else 0
# top locations
locations = [j["location"] for j in jobs if j.get("location")]
location_counts = Counter(locations).most_common(5)
return render_template("dashboard.html",
total_jobs=total_jobs,
skill_counts=skill_counts,
avg_salary=avg_salary,
location_counts=location_counts,
jobs=jobs[:5]
)
```

5.2 How the Analytics Work

Metric	How it is calculated
total_jobs	len(jobs) — simple count of all rows returned from Supabase

avg_salary	Sum of all salary_min values divided by count, rounded to nearest integer
skill_counts	Counter flattens all skills arrays into one list, counts occurrences, returns top 10
location_counts	Counter counts location strings, returns top 5 most common

5.3 Jinja2 Template Syntax

Jinja2 is Flask's built-in template engine. It lets you embed Python variables and logic directly in HTML using special tags:

Syntax	What it does	Example
{{ variable }}	Outputs a variable value	{{ total_jobs }} → 60
{% for x in list %}	Loops over a list	{% for skill, count in skill_counts %}
{% endfor %}	Ends a for loop	{% endfor %}
{{ value | int }}	Applies a filter — converts to int	{{ job.salary_min | int }} → 64487
{{ value | join(', ') }}	Joins a list into a string	{{ job.skills | join(', ') }}
{{ "x" if condition }}	Inline conditional	{{ job.salary_min if job.salary_min else "N/A" }}

ERROR ENCOUNTERED & FIX

Error:

```
jinja2.exceptions.TemplateSyntaxError: expected token "end of print statement", got ":"
```

Cause:

Used Python f-string format syntax inside Jinja2: {{ avg_salary:, }} — the :, number formatting is Python only, not supported in Jinja2.

Fix:

```
{{ "{:,}".format(avg_salary) }} — call Python's .format() method inside the template instead.
```

5.4 dashboard.html — What It Renders

Section	Data source	What user sees
Overview stats	total_jobs, avg_salary, skill_counts	60 jobs, £64,937 avg salary, top skill
Top Skills list	skill_counts (list of tuples)	machine learning — 25 jobs, python — 22 jobs...
Top Locations list	location_counts (list of tuples)	London UK — 41 jobs, The City — 16 jobs...
Recent Jobs	jobs[:5] (first 5 records)	5 job cards with title, company, salary, skills, link

5.5 Actual Output Confirmed in Browser

Metric	Value
Total Jobs Tracked	60
Average Salary	£64,937
Top Skill	machine learning (25 jobs)
2nd Skill	python (22 jobs)
3rd Skill	data science (10 jobs)
Top Location	London, UK (41 jobs)
2nd Location	The City, Central London (16 jobs)

Result: ✓ Full data pipeline confirmed working — Adzuna API → Pandas → Supabase → Flask → Browser.

5.6 Full Data Flow Summary

Step	File	What happens

1. Fetch	scraper/fetch_jobs.py	Calls Adzuna API, returns raw JSON list
2. Clean	scraper/clean_data.py	Pandas normalizes fields, extracts skills
3. Store	scraper/store_jobs.py	Inserts clean rows into Supabase jobs table
4. Query	app.py dashboard()	Fetches all jobs from Supabase on page load
5. Compute	app.py dashboard()	Counter calculates skill/location stats
6. Render	templates/dashboard.html	Jinja2 injects data into HTML
7. Display	Browser	User sees live stats and job listings

Build Progress

Step	Description	Status
1	Project setup — venv, packages, file structure, local Flask server	✓ Done
2	Adzuna API — register, get keys, write fetch_jobs.py, pull live data	✓ Done
3	Data cleaning — Pandas, normalize fields, extract skill tags	✓ Done
4	Supabase — create jobs table, store first batch of 60 jobs	✓ Done
5	Dashboard route — query Supabase, compute analytics, render template	✓ Done
6	Plotly charts — skills, location, salary interactive charts	Next →
7	Jobs listing page — /jobs route with filters	
8	UI polish — dark CSS theme	
9	GitHub + Railway deployment	
10	Add to portfolio via /admin	
11	(Stretch) ML salary prediction model	
